Module 14: SOQL & SOSL

```
In any application, we have 3 layers:
1. User Interface --> Front end
2. Logic --> Controller
3. Data Model --> Backend --> The layer which interacts with the Database.
Here we use SOQL, SOSL, DML, Apex Triggers
```

14.1 SOQL

Definition

SOQL abbreviated for Salesforce Object Query Language, is instruction given to Salesforce Database to retrieve the required data.

- **Syntax:**

  SELECT <field1, field2,...> FROM <ObjectAPIName>

  [WHERE conditionExpression]

  [LIMIT numberOfRows]

  [other options such as ORDER BY, GROUP BY]

Here SELECT and FROM are the mandatory keywords used to complete the instructions

– Others are optional keywords which are used as per desired output

SOQL can be applied on Standard, Custom, External Objects and Platform Events

14.1 SOQL

SQL vs SOQL

SQL	SOQL
SQL – Structured Query Language	SOQL – Salesforce Object Query Language
It is widely used in many languages	It is used to query data only from Salesforce
Support statements for querying, transaction control, schema definition, CRUD and more	Supports only for query statements
Support SELECT *	Does not support SELECT * But supports SELECT FIELDS(ALL)
Support JOINS which are written using 'LEFT' and 'RIGHT' keywords	Supports RELATIONSHIP QUERIES, which are written using parent-child, child-parent queries
Are not governed by any limits	It supports multi-tenant, therefore its governed by limits

14.1 SOQL

Example

Write a SOQL Query to provide the list of Accounts with Name

- SELECT Name FROM Account

Write a SOQL Query to provide the list of Positions with Name and Department

- SELECT Name, Department__c FROM Position__c

Main Return Types in SOQL

1. List<sObject>: This is the most common return type. It returns a collection of Salesforce objects matching your query criteria. The sObject represents the specific type of Salesforce object you're querying, like Account, Contact, or Position__c.

2. sObject: Can assign a SOQL query to a single sObject variable. However, this is only recommended if expectation is exactly one result. If there are no matches or multiple matches, it will encounter an exception during execution.

3. Integer: This return type is specific to SOQL queries that use the COUNT function. It returns the number of records that meet the query criteria.

4. AggregateResult: This return type is used for SOQL queries with aggregate functions like SUM, COUNT, or AVG. It provides a structured representation of the aggregated data, allows to access details like the field name, the calculated value, and more.

WHERE keyword Operators

WHERE keyword Operator	Usage
=	Equals
!=	Not Equals
>	Greater than
<	Lesser than
>=	Greater than or equals to
<=	Lesser than or equals to
LIKE	Acts as wildcard in String fields: '%' matches 0 or many characters '_' matches 1 character
IN / NOT IN	Value equals/doesn't equal any one of the values
INCLUDES / EXCLUDES	Value equals/doesn't equal multi – select picklist values
AND	Acts as Logical AND Operator
OR	Acts as Logical OR Operator
NOT	Acts as Logical NOT Operator

Examples of WHERE keyword in SOQL

Filter Query Results with Conditions:

Examples:

Display AnnualRevenue and Industry of 'Global Media' Account

- SELECT AnnualRevenue, Industry FROM Account WHERE Name = 'Global Media'

Display Names and Departments of all Positions excluding 'IT' Department

- SELECT Name, Department__c FROM Position__c WHERE Department__c != 'IT'

Display First Name and Last Name of Candidates who are having experience between 5 to 10 years

- SELECT First_Name__c, Last_Name__c FROM Candidate__c
  WHERE Years_of_Experience__c > 5 AND Years_of_Experience__c < 10

Display Names of Positions containing 'UI' word in the name

- SELECT Name FROM Position__c WHERE Name LIKE '%UI%'

Display the Name and Status fields of Job Applications with 'Open' or 'Hold' status

- SELECT Name, Status__c FROM Job_Application__c WHERE Status__c IN ('Open','Hold')

Display the list of Candidates with First Name, Last Name and Education. Education should be either 'BA / BS' or 'MA / MS / MBA'

- SELECT First_Name__c, Last_Name__c, Education__c FROM Candidate__c WHERE
  Education__c INCLUDES ('BA / BS','MA / MS / MBA')

ORDER BY - Definition

- The ORDER BY clause control the order in which the query results are returned.
- It helps to sort by any field in the object queried. Just specify the field name after ORDER BY.
- By default, sorting is ascending (A to Z, lowest to highest).

When using ORDER BY there are additional conditioned keywords that can be used in SOQL.

- ASC
- DESC
- NULLS FIRST
- NULLS LAST

ORDER BY keyword examples in SOQL

Display list of Accounts with Name in Ascending order

- SELECT Name FROM Account ORDER BY Name [ASC]

Display list of Offers with Actual Salary in Descending order

- SELECT Name, Actual_Salary__c FROM Offer__c ORDER BY Actual_Salary__c DESC

Display Review object Experience Comments in order of empty comments to filled comments

- SELECT Name, Experience_Comments__c FROM Review__c
  ORDER BY Experience_Comments__c NULLS FIRST

Display Position records in order of provided closed date

- SELECT Name, Date_Closed__c FROM Position__c ORDER BY Date_Closed__c NULLS LAST

Examples of LIMIT keyword in SOQL

The LIMIT keyword in SOQL is used to specify the maximum number of records to be returned by a query. It helps to restrict the result set to a manageable size, especially when dealing with large datasets.

Examples:

Show 5 Opportunities ordered by Amount in descending order

- SELECT Name, Amount FROM Opportunity ORDER BY Amount DESC LIMIT 5

Show 10 Candidates ordered by Years of Experience

- SELECT Name, Years_of_Experience__c FROM Candidate__c ORDER BY Years_of_Experience__c LIMIT 10

COUNT keyword definition & examples in SOQL

- The COUNT() function retrieves the total number of records matching the specified criteria in the query.
- COUNT(fieldname) function counts the number of non-null (not empty) values for a specific field across the retrieved records.

Examples:

Display the count of Prospect Account records

- SELECT COUNT() FROM Account WHERE Type = 'Prospect'

Display the count of Contacts which are associated with Account

- SELECT COUNT(AccountID) FROM Contact

Display the count of Candidates who is having more than 5 years of experience

- SELECT COUNT() FROM Candidate__c WHERE Years_of_Experience__c >5

```
Standard objects:
Account --> Parent --> One
Contact --> Child --> many

One account can have multiple contacts

first account --> 3 contacts associated
second account --> 5 contacts associated
third account --> 2 contacts associated

Aggregate functions: These functions are used with group by clause.
1. Count()
2. Sum()
3. min()
4. max()
5. average()
```

```
ID is the field present in all the objects(standard or custom). It stores the
record ID. It is unique. We also consider it as a Primary Key.

Account --> ID --> primary key
Contact --> ID --> Primary key
position__c --> ID --> primary key

select Account.Name, count(ID) from contact group by Account.Name
```

Examples of GROUP BY keyword in SOQL

Using the GROUP BY clause in SOQL allows to perform aggregate functions and group records by specific fields. This is particularly useful for summarizing data, such as counting the number of records that share a common field value or calculating averages.

Examples:

Display list of Contacts associated with each Account (Query Editor)

- SELECT Account.Name, COUNT(Id) FROM Contact GROUP BY Account.Name

Display Sum of Annual Revenue for accounts grouped by Industry (Anonymous Window)

- List<AggregateResult> results = [SELECT Industry, SUM(AnnualRevenue) totalRevenue
  FROM Account GROUP BY Industry];

```
for (AggregateResult ar : results) {
    String industry = (String) ar.get('Industry');
    Decimal totalRevenue = (Decimal) ar.get('totalRevenue');
    System.debug('Industry: ' + industry + ', Total Revenue: ' + totalRevenue);
```

Implementing SOQL in Apex

- SOQL fetches multiple records
- We must store the collected data in a variable
- We use collection datatype to store multiple records which is list/array of sObject
- We use sObject datatype to store single record

Examples:

Display list of Candidates with First Name & Last Name

- List<Candidate__c> candList = [SELECT First_Name__c, Last_Name__c FROM Candidate__c]; //SOQL returning Multiple Records

Display Account Information of 'Global Media' with Annual Revenue and Number of Employees

- Account acct = [SELECT AnnualRevenue, NumberOfEmployees FROM Account WHERE Name='Global Media']; //SOQL returning Single Record

Access Variables in SOQL Queries

To bind a variable use colon(:) in the where condition of SOQL Query

Examples:

Retrieve High Priority Position records

- String priority = 'High';
  List<Position__c> pos = [SELECT Name, Priority__c FROM Position__c WHERE Priority__c = :priority];

Retrieve CloseDate for the Case Numbers '00001000', '00001001'

- Set<String> caseSet = new Set<String>{'00001000', '00001001'};
  List<Case> caseList = [SELECT ClosedDate FROM Case WHERE CaseNumber IN :caseSet];

14.1 SOQL

Display list of candidates from USA who are not currently employed

Solution:

```
List<Candidate__c> candList =[SELECT Name, Currently_Employed__c, Country__c FROM
        Candidate__c WHERE Currently_Employed__c = FALSE and Country__c = 'USA'];
System.debug('Candidates are '+candList);
```

Execution Log

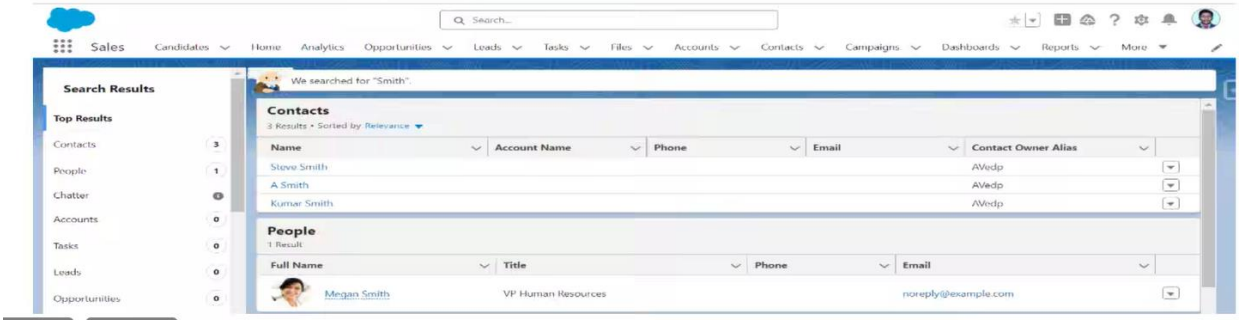
Timestamp	Event	Details
06:23:11:017	USER_DEBUG	[2]|DEBUG|Candidates are (Candidate__c:{Name=C-0006, Currently_Employed__c=false, Country__c=USA, Id=a01GB000031jumDYAQ})

SOSL

14.2 SOSL

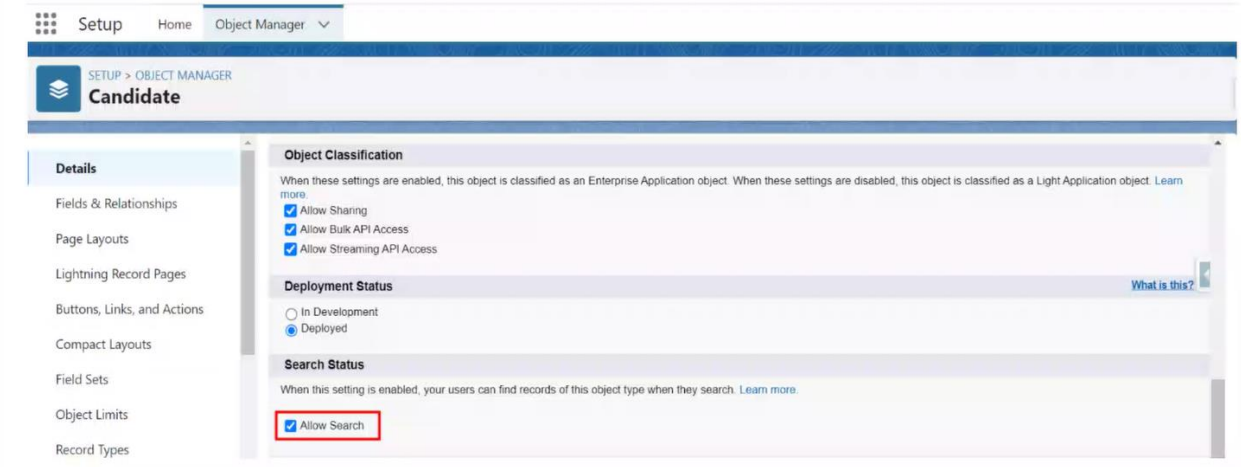
Definition & Use Case

- SOSL abbreviated for Salesforce Object Search Language
- Used to perform text search
- SOSL is used to search data/value in fields on multiple objects
- It's used to search record, but we don't know in which object or field the data resides
- It works like Salesforce Global Search



Allow Search

To allow search in any custom object need to enable Allow Search from Object Details



Syntax & Keywords

- **Syntax:** FIND 'data/value' IN Search-Group RETURNING Objects
  FIND is mandatory key word in SOSL, other key words are optional

- Search group can have below keys in which the data/value specified will be searched in:
  - ALL FIELDS
  - NAME FIELDS
  - EMAIL FIELDS
  - PHONE FIELDS

- The Search Query can be 2 types:
  - Single word - words are case insensitive
  - Phrases - collection of words and spaces within double quotes

Example in Query Editor and Apex

Query Editor:

FIND {<searchTerm>} [IN SearchGroup] [RETURNING <OBJECTAPI(FIELDS)>]

Example:

Find Jones and return respective Account Name & Contact First Name and Last Name

- FIND {Jones} IN ALL FIELDS RETURNING Account(Name),Contact(FirstName, LastName)

Anonymous Window:

FIND '<searchTerm>' [IN SearchGroup] [RETURNING <OBJECTAPI(FIELDS)>]

Example:

Find Jones and return respective Account Name and Department & Contact First Name and Last Name – using Anonymous Window

- List<List<sObject>> accConRec = [FIND 'Jones' IN ALL FIELDS RETURNING Account(Name, AccountNumber), Contact(FirstName, LastName)];

Search - Single Word and Phrases:

Search Query contains two types of text:

- Single Word - Bill

Examples:

- FIND 'Bill' (APEX)
- FIND {Bill} (Query Editor)

- Phrase - collection of words eg: "Nick Tran"

Example:

- FIND 'Nick Tran' (APEX)
- FIND {Nick Tran} (Query Editor)

Optional Keywords

Where:

RETURNING Account(Name, Industry WHERE Industry='Apparel')

Order By:

RETURNING Candidate__c(Name, Phone__c ORDER BY Country__c)

Limit:

RETURNING Position__c(Department__c, Status__c LIMIT 10)

Example:

Find 'Jo' in Candidates who are currently employed and display only first two records in alphabetical order.

- FIND {Jo*} IN ALL FIELDS RETURNING Candidate__c(First_Name__c, Last_Name__c WHERE Currently_Employed__c = TRUE ORDER BY First_Name__c LIMIT 2)

Example for SOSL within APEX

Find Account and Contact Records in Name fields having word 'Acme' or having six letter word having last four letters as 'ward'. (using APEX)

```
String nm = 'Acme OR ??ward';
List<List<sObject>> accConRec = [FIND :nm IN NAME FIELDS RETURNING Account(Name),
                                 Contact(LastName,FirstName)];
List<Account> accList= (List<Account>)accConRec[0];
List<Contact> conList= (List<Contact>)accConRec[1];
for(Account acc:accList){
   System.debug('Account is '+acc);
}
for(Contact con:conList){
   System.debug('Contact is '+con);
}
```

SOSL Best Practices

SOSL is a powerful tool for searching across multiple objects and fields.
Following best practices ensures efficient and optimized usage of SOSL queries.

1. Use specific fields in RETURNING Clause
2. Limit the number of returned records
3. Index searchable fields
4. Optimize search tokens
5. Use SOSL for broad searches
6. Use SOSL for full-text search

14.2 SOSL

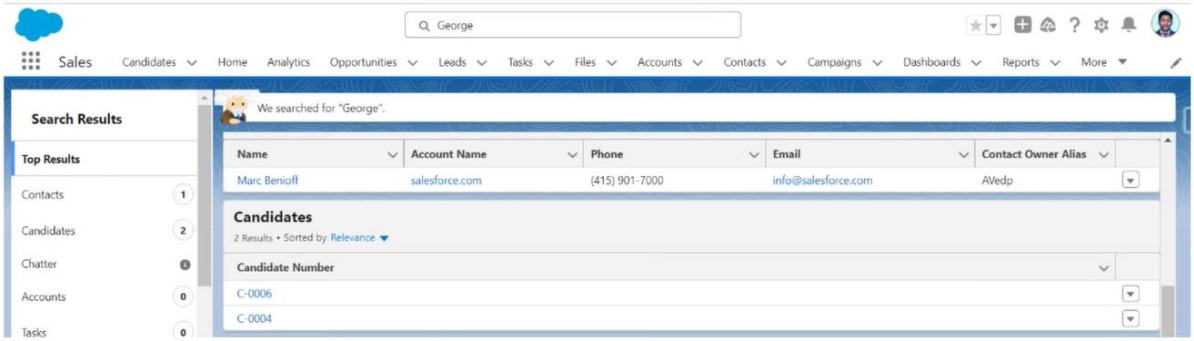
Find Position and Candidate records in all fields having words 'Manager' or 'Bill' using APEX

Solution:

```
String nm = 'Manager OR Bill';
List<List<sObject>> sobjList = [FIND :nm IN ALL FIELDS RETURNING Position__c(Name),
                        Candidate__c(Name)];
List<Position__c> posList= (List<Position__c>)sobjList[0];
List<Candidate__c> candList= (List<Candidate__c>)sobjList[1];
System.debug('Total number of Position__c Records is '+posList.size());
for(Position__c pos:posList){
   System.debug('Position__c is '+pos);
}
System.debug('Total number of Candidate Records is '+candList.size());
for(Candidate__c cand:candList){
   System.debug('Candidate is '+cand);
}
```

14.2 SOSL

Find "George" keyword in all fields returning Candidate and Contact records with Name. Check whether it gives same result as below,



SOQL vs SOSL

SOQL	SOSL
Salesforce Object Query Language	Salesforce Object Search Language
Only one object at a time can be searched (Search in Single object)	Many object can be searched at a time (Search in entire organization or Database)
Query all type of fields	Query on only email, text or phone fields
It can be used in classes and triggers	It can be used in classes but not in trigger
DML Operation can be performed on query results	DML Operation cannot be performed on search results
SOQL can be used when we know in which objects or fields the data resides	SOSL can be used when we don't know in which object or field the data resides.
Can retrieve data from single object or multiple objects that are related to each other	Can retrieve multiple objects and field values where the objects may or may not be related to each other

14.3

Manipulate Records with DML

Data Manipulation Language (DML)

- DML is used to Insert, Update, Delete, Undelete, Upsert and Merge records.
- DML operations are crucial for building dynamic and data-driven Salesforce applications.
- They allow you to automate tasks, synchronize data, and implement custom business logic within your Salesforce org.

INSERT	UPDATE	DELETE	UNDELETE	UPSERT
The insert DML operation adds one or more SObject records into your organization's data.	The update DML operation updates one or more sObject records in your organization's data.	The delete DML operation deletes one or more existing records.	The undelete DML operation restores one or more existing sObject records, from your organization's Recycle Bin.	The upsert DML operation creates new sObject records and/or updates existing sObject records within a single Statement.

14.3 Manipulate Records with DML

Insert Operation

```apex
public class DmlOperation
{
    public void insertAccountRecord()
    {
        Account acc = new Account();
        acc.Name='Joe Sim';
        acc.Type='Customer';
        acc.Industry='Banking';
        acc.Description='New Customer';
        insert acc;
    }
}
```

```apex
public class DmlOperations
{
    public void insertCandidateRecord()
    {
        Candidate__c cd = new Candidate__c();
        cd.First_Name__c='Harriet';
        cd.Last_Name__c='Campbell';
        cd.Phone__c='9999988888';
        cd.Gender__c='Female';
        insert cd;
    }
}
```

DML --> Data Manipulation Language

We have 6 DML statements in Apex:

1. Insert
2. Update
3. Delete
4. Upsert
5. Merge
6. Undelete

WE have 4 types of DML:
1. Single DML --> To insert, update or delete single record at a time.
2. Bulk DML in full transaction mode --> to insert, update or delete multiple records at a time. (if there are 20 records to be inserted, then either all 20 will be inserted or none of the records will be inserted)
3. Bulk DML in partial transaction mode --> to insert, update or delete multiple records at a time. (if there are 20 records to be inserted, then partial transaction is allowed)
4. Related DML -->

To-Do

14.3 Manipulate Records with DML

Problem Statement:
The company is currently in the midst of a recruitment process, with several candidates at various stages of consideration for positions within the IT department.
However, due to unforeseen organizational changes, there is now a need to halt this recruitment process and close all the open IT positions. Write a Class using DML operation to achieve the same.

Bulk DML

It is recommended to perform DML operations in bulk to avoid hitting government limits

```
public class BulkDML
{
    public static void insertAccountData()
    {
        List<Account> accountList = new List<Account>();
        for(Integer i=0; i<10; i++) // i=0 to 9 => 10 times
        {
            Account acc = new Account();
            acc.Name='Looping '+i;
            accountList.add(acc);
        }
        insert accountList;
    }
}
```

Database Methods

- In Apex programming, the Database class provides a suite of static methods that allow developers to perform DML (Data Manipulation Language) operations. These methods offer more flexibility and control compared to traditional DML statements, as they can handle partial successes and provide detailed result information.

- Here are some of the key methods provided by the Database class:
  1. Database.insert(CollectionVariable, allOrNone) -> Database.SaveResult[]
  2. Database.update(CollectionVariable, allOrNone) -> Database.SaveResult[]
  3. Database.delete(CollectionVariable, allOrNone) -> Database.DeleteResult[]
  4. Database.upsert(CollectionVariable, allOrNone) -> Database.UpsertResult[]

Insert Records with Partial Success

```
public class PartialTransactionDML
{
   public void insertPosition()
   {
       List<Position__c> posList = new List<Position__c>();
       Position__c pos1 = new Position__c(Name='SF Admin',Job_Description__c='Should be aware about basics');
       Position__c pos2 = new Position__c(Name='SF Support');  // Job Description required field is missing
       posList.add(pos1);
       posList.add(pos2);
       Database.SaveResult[] resList = Database.insert(posList, false);

       for(Database.SaveResult sResult : resList)
       {
           if(sResult.isSuccess())
              System.debug('Id of inserted record: '+sResult.getID());
           else
           {
               for(Database.Error err : sResult.getErrors())
               {
                    System.debug('Error Message: '+err.getMessage());
                    System.debug('Fields affected: '+err.getFields());
               }
            }
       }
   }
}
```

Upsert Program

```
public class UpsertOperation
{
    public void upsertAccount()
    {
        List<Account> accountList = new List<Account>();

        // updating exsisting record
        Account existingAcc = [SELECT Name, Type FROM Account WHERE Name='Global Media'];
        existingAcc.Type='Customer';

        // Insert new account
        Account newAcc = new Account(Name='Aryan', Industry='Banking', Type='Other');

        // added to list and performed upsert operation
        accountList.add(existingAcc);   // adding existing record for update
        accountList.add(newAcc);   // adding new record for insert
        upsert(accountList);  // upsert = update + insert
    }
}
```

Example 1 : Without Exception Handling

```
public class WithoutExceptionHandling
{
    public void insertCandidate()
    {
       Candidate__c can = new Candidate__c();
       can.First_Name__c = 'Prayag';
       can.Last_Name__c =  'Raj';
       can.State_Province__c = 'Maharashtra';
       insert can;
    }
}
```

14.4

Execution Governors & Limits

Introduction

Governor limits in Salesforce are used to ensure the effective use of resources on the multi-tenant platform

To execute the code efficiently, Salesforce.com sets some limits

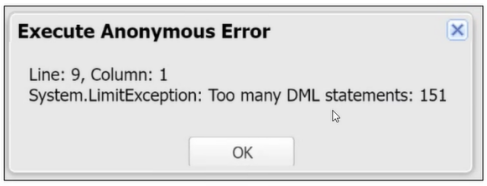
Per-Transaction Apex Limits

Description	Synchronous	Asynchronous
Total number of SOQL queries issued	100	200
Total number of SOSL queries issued	20	20
Total number of DML statements issued	150	150
Total number of records retrieved by a single SOSL query	2,000	2,000
Total number of records retrieved by SOQL queries	50,000	50,000
Total number of records retrieved by Database.getQueryLocator	10,000	10,000
Salesforce Governor Limits for heap size	6 MB	12 MB
Maximum CPU time on the Salesforce servers	10,000 ms	60,000 ms

Example : Governor Limit (DML Limit Exception)

```
public class GovernorLimit
{
    public void dmlLimit()
    {
        for(Integer i=0; i<151; i++)
        {
            Account acc = new Account();
            acc.Name = 'Account'+i;
            insert acc;
        }
        System.debug('Account Records Got Inserted');
    }
}
```

Output



Example : Handling Governor Limit (DML Limit Exception)

```
public class GovernorLimit
{
    public void dmlLimit()
    {
        List<Account> accountList = new List<Account>();
        for(Integer i=0; i<151; i++)
        {
            Account acc = new Account();
            acc.Name = 'Account'+i;
            accountList.add(acc);
        }
        insert accountList;
        System.debug(accountList.size() +' Account Records Got Inserted');
    }
}
```

Output

Execution Log

Timestamp	Event	Details
21:17:57:739	USER_DEBUG	[13]|DEBUG|151 Account Records Got Inserted